

UNIVERSITY OF LEICESTER

School of Computing and Mathematical Sciences

CO7201 Individual Project

Final Report

Implementation of an SOS-based Interpreter for the L Programming Language

Nana Kwadwo Adjei Manu
nkam1@student.le.ac.uk
239063918

Project Supervisor: Dr Paula Severi
Principal Marker: Dr Anand Sengodan

Word Count: 7902

May 15, 2025

Abstract

This report describes the design, implementation, and evaluation of an interpreter for a functional programming language, called L, based on the untyped lambda calculus, extended with common features. The primary objective was to explore the core concepts of language implementation, including parsing, evaluation, environment management, and formal semantics, using the Functional Programming Language Haskell. Central to this work was the formal description of L using Structured Operational Semantics (SOS), providing an account of its behavior. The project explored implementing integer and boolean types with associated operations (arithmetic, equality, conditionals), list manipulation primitives (cons, head, tail, isEmpty, literal lists), lexical scoping via let bindings, and support for top-level, mutually recursive function definitions. A custom parser combinator library was developed for syntax analysis. The evaluation strategy employs an environment model to handle variable bindings and closures for lexical scoping, translating the high-level SOS rules into executable interpreter code. This report evaluates the achieved results, critically compares the implementation choices (e.g., custom parser, implicit recursion) with related work, and discusses future extensions.

DECLARATION:
All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.
Name: Nana Kwadwo Adjei Manu
Date: April 28, 2025

Contents

1 Introduction **5**
- 1.1 Background and Motivation . . . 5
- 1.2 Aims & Objectives . . . 5
- 1.3 Report Structure . . . 6

2 Literature Review **6**
- 2.1 Formal Semantics Overview . . . 6
- 2.2 Structured Operational Semantics . . . 7
  - 2.2.1 Origins and Definition . . . 7
  - 2.2.2 Core Concepts . . . 7
  - 2.2.3 Small-step vs. Big-step (Natural) Semantics . . . 8
- 2.3 Review of Lambda Calculus . . . 9
  - 2.3.1 Evaluation Strategies . . . 9
  - 2.3.2 Comparative Studies . . . 9
- 2.4 Haskell for Language Implementation . . . 9
- 2.5 Parsing Techniques . . . 10

3 Requirements **10**
- 3.1 Functional Requirements . . . 10
- 3.2 Non-Functional Requirements . . . 10

4 Syntax & Semantics of *L* **10**
- 4.1 Overview . . . 10
- 4.2 SOS Definition for L . . . 10
  - 4.2.1 BNF Grammar for L . . . 11
  - 4.2.2 Operational Semantics of L . . . 12

5 Design & Implementation **14**
- 5.1 Interpreter Architecture Overview . . . 14
- 5.2 Viability of Haskell as an implementation language . . . 15
- 5.3 Ast Implementation . . . 15
  - 5.3.1 Abstract Syntax Tree for L . . . 15
- 5.4 Data types & Values . . . 16
- 5.5 Parser Implementation . . . 16
  - 5.5.1 Parser Design (`Parser.hs`) . . . 17
  - 5.5.2 Choice of Parsing Technique . . . 17
  - 5.5.3 Structure of the Parser Module . . . 17
  - 5.5.4 Parsing Lambdas (`parseLambda`) . . . 17
- 5.6 Interpreter Implementation . . . 18
  - 5.6.1 Evaluator Design (`Evaluator.hs`) . . . 19
  - 5.6.2 Main Evaluation Function Signature . . . 19
  - 5.6.3 Environment Representation (`Env`) . . . 19
  - 5.6.4 Evaluation of Lambda Abstractions (`Lam var body`) . . . 19
  - 5.6.5 Evaluation of Function Application (`App fun arg`) . . . 19
  - 5.6.6 Scope Management for Let Expressions and Function Parameters . . . 20
  - 5.6.7 Function Parameters: . . . 20
- 5.7 User Interface . . . 23
  - 5.7.1 Evaluating L programs via the REPL . . . 23
  - 5.7.2 Evaluating L programs via Web interface . . . 24

6 Implementation Details **26**
6.1 Module Responsibilities . . . 26
6.1.1 `Ast.hs` (Abstract Syntax Tree) . . . 26
6.1.2 `Value.hs` (Runtime Values and Environment) . . . 26
6.1.3 `Parser.hs` (Source Code Parsing) . . . 26
6.1.4 `Evaluator.hs` (Expression Evaluation) . . . 26
6.1.5 `Main.hs` (REPL and Web Interface) . . . 27
6.2 Flow of Data and Interaction . . . 27
6.2.1 Backend data flow . . . 27
6.2.2 Frontend Implementation and Data Flow (React) . . . 27
6.2.3 App.tsx (Main Application Component) . . . 27
6.3 Tools and Technologies Used . . . 29
6.3.1 Interpreter Backend (Core Language Implementation) . . . 29
6.3.2 Primary Language . . . 29
6.3.3 Build System and Package Management . . . 29
6.3.4 Key Haskell Libraries . . . 30
6.3.5 Frontend (Web Interface) . . . 30
6.3.6 Version Control and Collaboration . . . 30

7 Testing and Evaluation **31**
7.1 Testing Strategy . . . 31
7.2 Test Coverage . . . 31
7.2.1 Coverage results . . . 33
7.3 Evaluation of Results . . . 34
7.3.1 Comparison with related work . . . 34
7.4 Limitations . . . 35

8 Conclusion **35**
8.1 Reflections . . . 36
8.1.1 What went well? . . . 36
8.1.2 What could have been done differently? . . . 36
8.1.3 Comparison with objectives . . . 36
8.1.4 Lessons Learned . . . 36
8.2 Future Work . . . 37

9 References **37**

1 Introduction

The aim of this project is to study a simple high-level programming language, referred to as L, by defining its syntax and semantics using Structured Operational Semantics (SOS) and implementing an SOS-interpreter in Haskell. SOS provides a formal method to describe the execution behavior of programming languages without going into low-level machine details [1]. This report represents a significant work, detailing the background, motivation, literature review, and requirements, while outlining the structure of the final report.

1.1 Background and Motivation

Structural Operational Semantics (SOS), introduced by Gordon Plotkin in 1981, provides a rigorous framework for formalizing programming language behavior through inference rules that define step-by-step execution [1]. At its core, SOS uses transition relations of the form $\langle e, \sigma \rangle \rightarrow \langle e', \sigma' \rangle$ to represent how program expressions and states evolve during computation [9]. Through its compositional nature, language semantics can be built from smaller rules, each following the format of premises above a horizontal line and a conclusion below [10]. SOS exists in several variants: small-step semantics for modeling individual computation steps, big-step semantics (natural semantics) for relating expressions directly to final values, and labeled transition systems for representing observable behaviors [11, 12]. The framework has proven invaluable for language design, compiler verification, type system formalization, and concurrency modeling [13]. Recent developments such as modular SOS and component-based semantics enhance the flexibility of the SOS framework while maintaining its mathematical rigor [3]. Despite the newer semantic approaches, SOS remains fundamental to programming language theory due to its clarity, precision, and effectiveness in reasoning about program properties.

High-level programming languages such as Haskell, Java, and Python abstract away low-level machine details, enabling developers to focus on logic and functionality. Understanding how these languages are defined and executed is crucial for language design and implementation. SOS, introduced by Plotkin [1], offers a systematic approach to specify both syntax and semantics, effectively acting as a blueprint for an interpreter. The motivation for this project lies in exploring how SOS can be used to define a simple language L and translating this formal specification into a functional interpreter using Haskell, a language well suited for such tasks due to its strong type system and functional paradigm [5].

1.2 Aims & Objectives

- **Define Language Syntax & Semantics**: Specify the grammar and operational semantics for the target language, *L*, using the small-step style of Structured Operational Semantics (SOS). This involves the creation of formal inference rules that govern the execution of the constructs within the L language including data types (integers, booleans, lists, closures) and core expressions (literals, arithmetic operations, boolean operations, variables, let bindings, lambda abstractions (Lam), function application (App), if-then-else conditionals, list operations)

- **Develop a Parser**: Implement a parser that transforms source code text into an Abstract Syntax Tree (AST), correctly handling function definitions and applications.

- **SOS Interpreter Implementation**: Create an evaluation function that takes an AST node and an environment, and computes the corresponding value, strictly adheres to the behavior defined by the SOS rules. This implementation serves not only as an execution engine but also as a practical validation of the formal semantic definition. This includes handling variable lookups, creating closures for lambda abstractions, applying closures to arguments according to lexical scope rules, and evaluating control flow structures.

- **Tool Creation**: To produce a usable software artifact, the SOS-based interpreter equipped with a user interface, that serves as a demonstration and experimental platform for the formally defined semantics of L. This includes a very simple web-based interface and its accompanying Read Evaluate Print Loop (REPL) to write simple L programs, evaluate and return the expected outputs based on the defined semantics of L.
- **Error Handling**: Incorporate robust error handling for both parse-time and run-time errors (e.g., type mismatches, undefined variables, incorrect function arity if checked).
- **Testing**: Develop a comprehensive set of unit tests to verify the correctness of the parser and evaluator for all language constructs, including the definition and application of functions.
- **Reflection**: Critically evaluate the design choices, implementation and the capabilities of the final interpreter.

1.3 Report Structure

This report is organized as follows: Section 2 provides a background on language interpreters, functional programming concepts relevant to interpretation, and related work. Section 3 details the design of our language and its interpreter, covering the AST, parser, value representation (including closures), and evaluator. Section 4 discusses the implementation specifics, including key code excerpts related to function handling and challenges encountered. Section 5 presents the test strategy and interpreter evaluation. Finally, Section 6 offers concluding remarks, reflects on the project, and suggests future work.

2 Literature Review

This section reviews key literature relevant to SOS, programming language semantics, lambda calculus as a model for computation, and Haskell-based interpreter implementation.

2.1 Formal Semantics Overview

Understanding the behavior of computer programs requires a precise definition of the meaning, or semantics, of the programming language constructs they employ. Although informal descriptions in natural language or reference implementations (such as compilers or interpreters) are common, they often suffer from ambiguity or may contain bugs, leading to inconsistencies and difficulties in rigorous analysis [9]. For example, the evaluation order of the expressions with side effects in C can be unspecified, leading to different results between compilers [22]. Similarly, relying solely on a definitional interpreter means that any bugs or quirks in the interpreter become, by definition, part of the language semantics, which is often undesirable. Formal semantics aims to overcome these limitations by using mathematically based techniques and precisely defined notation to describe the meaning of programming languages [20]. This enables unambiguous specification, formal reasoning about program properties, verification of implementations, and a deeper understanding of language design choices. Several distinct approaches to formal semantics have been developed, each offering different perspectives and suitable for different purposes [10]:

- **Operational Semantics**: This approach defines the meaning of a program by specifying the sequence of computational steps it takes during execution on an idealized or abstract machine [20]. It focuses on how a program executes. This style is particularly well-suited for defining interpreters, analyzing computational complexity, and reasoning about runtime behavior. Structural Operational Semantics (SOS), the focus of this project, is a prominent style within this category.

- **Denotational Semantics**: Here, the meaning of each program phrase is defined by mapping it to an abstract mathematical object, such as a function within a suitable mathematical domain (e.g., Scott domains) [10]. The meaning of a compound phrase is defined compositionally from the meanings of its sub-phrases. This approach excels in providing abstract, implementation-independent meanings and supporting compositional reasoning about program equivalence.
- **Axiomatic Semantics**: This method defines semantics using logical axioms and inference rules (e.g. Hoare logic) that relate program statements to logical assertions (preconditions and postconditions) about the program state [10]. It focuses on what properties hold true before and after the execution of program fragments, making it highly suitable for formal program verification.

Foundational texts, such as Nielson and Nielson's Semantics with Applications [10], provide comprehensive introductions to these diverse approaches and explore their interrelationships and applications in computer science.

2.2 Structured Operational Semantics

Plotkin's work on SOS [1] introduced a framework to define the operational semantics of programming languages using transition rules. These rules describe how program states evolve, providing a clear and modular approach to semantics specification. Subsequent extensions, such as those of Hennessy [2], refined SOS for concurrent systems, though this project focuses on sequential execution for simplicity.

2.2.1 Origins and Definition

Structural Operational Semantics (SOS) represents a particular and highly influential style of operational semantics introduced by Gordon Plotkin in the late 1970s and early 1980s [19]. His seminal work, "A Structural Approach to Operational Semantics," established the foundations for this method [19]. The core idea of SOS is to define the behavior of a programming language construct based on the behavior of its constituent parts, using a set of syntax-directed inference rules. The term "structural" emphasizes that the rules are defined inductively over the abstract syntax structure of the language [19]. Plotkin's aim was to provide a method that was simple, direct, and yielded concise, comprehensible semantic definitions, effectively formalizing the often informal, natural language descriptions accompanying programming languages [1]. SOS definitions are intended to be intuitive, relying on "symbol-pushing" rules that describe transitions between program states.

2.2.2 Core Concepts

The SOS framework is built upon a few fundamental concepts:

- **Configurations (γ)**: A configuration represents a snapshot of the state of computation at a specific moment. It typically includes the program fragment currently under consideration and any relevant contextual information, such as memory state or environment [18]. For a simple imperative language, a configuration could be a pair $\langle c,\sigma \rangle$, where c is the command (statement) being executed and σ is the current store (a list of mappings from variables to values). Configurations can be intermediate, representing an ongoing computation, or terminal (final), representing a completed computation or a resulting value [18].

- **Transition Relation ($\rightarrow$)**: This is a binary relation between configurations, denoted $\gamma \rightarrow \gamma'$, signifying that configuration γ can evolve into configuration γ in a single computational step [1]. This relation formally defines the dynamics of the language. Although related to the concept of abstract machines [18], SOS often abstracts away the machine details, focusing directly on the transitions between configurations defined by the program structure [18]. The behavior of a system is viewed as the sum of its possible transitions [1].

- **Inference Rules**: The transition relation $\rightarrow$ is defined inductively using a set of inference rules. These rules typically have the following form:

$$\frac{\text{Premise}_1 \quad \text{Premise}_2 \quad \ldots \quad \text{Premise}_n}{\text{Conclusion}} \quad \text{(Side Condition)}$$

  This rule states that if all the premises are satisfied (and the optional side condition is met), then the conclusion holds [18]. The premises often describe transitions of subcomponents of the construct in the conclusion. Rules without premises are called axioms and represent fundamental computational steps [21]. Rules are directed to syntax, which means that there is usually a set of rules associated with each syntactic construct of the language [18].

2.2.3 Small-step vs. Big-step (Natural) Semantics

Within operational semantics, a crucial distinction exists between small-step and big-step styles. SOS, as originally presented by Plotkin and used in this project, is primarily a small-step (also called transitional or reduction) semantics [18].

- **Small-step (SOS)**: Focuses on defining the relation $\rightarrow$, which describes the individual steps of computation. The meaning of a program is given by the sequence of transitions that starts with an initial configuration [18], explicitly modeling intermediate states.

- **Big-step (Natural Semantics)**: Focuses on defining a relation, often denoted $\Downarrow$ or $\rightarrow$, which describes the overall result of evaluating a construct. A rule like $<e, \sigma> v$ means expression e in store σ evaluates completely to value v in one conceptual "big step" [18], thus abstracting the intermediate computational steps.

Small-step semantics offers several advantages, particularly relevant to this project:

- **Modeling Intermediate States**: It provides a detailed view of the execution process, step-by-step, which is valuable for debugging and understanding program behavior.

- **Handling Non-termination**: Infinite computations are naturally represented as infinite sequences of ß transitions [18]. Big-step semantics typically only defines results for terminating computations.

- **Concurrency and Nondeterminism**: Modeling interleaving or parallel execution is often more straightforward with small steps, as individual steps from different threads or processes can be interleaved.

The choice of small-step SOS for this project aligns with the goal of implementing an interpreter, as the step-by-step transitions map naturally to the interpreter's evaluation loop. This choice also facilitates a detailed understanding of L's execution flow and provides a foundation suitable for potentially modeling more complex features in the future.

Mosses [3] emphasizes modular semantics, where language constructs are defined independently, facilitating extensibility. This is particularly relevant for defining L, as it allows for the incremental development of the language's features. Furthermore, Winskel [4] provides a comprehensive overview of formal semantics, including operational and denotational approaches, positioning SOS as a practical choice for interpreter implementation.

SOS has proven to be a versatile and widely used formalism beyond the basic definition of languages. Its applications include:

- **Language Specification**: Providing precise definitions for various programming languages [1].
- **Type Systems**: Proving type safety properties, such as Progress (a well-typed term is either a value or can take a step) and Preservation (a step preserves the type) [9]. An interpreter implementing operational semantics can serve as a foundation for exploring these properties.

2.3 Review of Lambda Calculus

All functional programming languages fundamentally have the basic features of composition and function abstraction of a single variable. The lambda calculus, often called the "assembly language" of functional programming, forms the basis of many functional languages like Haskell and Standard ML and consists of 3 very simple terms and all valid recursive combinations:

Lambda calculus, introduced by Alonzo Church in the 1930s, provides a minimalist formalism for defining and applying functions [23]. Its evaluation mechanisms have influenced the design of functional programming languages and the development of formal semantics in computer science. The choice of evaluation strategy affects termination, performance, and the expressiveness of programming languages.

2.3.1 Evaluation Strategies

Evaluation strategies define how and when expressions in lambda calculus are reduced. The most studied strategies include normal-order, applicative-order (call-by-value), and call-by-name.

- **Normal-Order Evaluation**: Normal-order evaluation always reduces the leftmost, outermost redex first. It is guaranteed to find a normal form if one exists [24]. This property makes it suitable for proof systems and theoretical models, though it can be inefficient due to repeated evaluations.
- **Call-by-Value**: In call-by-value, arguments are evaluated before being passed into functions. This strategy, commonly used in strict languages such as OCaml and Java, avoids redundant computation but can prevent termination in expressions that would succeed under normal-order [25].
- **Call-by-Name and Lazy Evaluation**: Call-by-name delays argument evaluation until needed. Although not commonly implemented directly, it inspired lazy evaluation used in languages like Haskell [26]. Lazy evaluation improves modularity and allows infinite data structures, but introduces challenges in implementation.

2.3.2 Comparative Studies

Plotkin's operational semantics formalize the distinction between call-by-name and call-by-value, showing their behavioral implications [25]. Ariola and Felleisen further explore the semantics of call-by-need—an optimization of call-by-name that avoids redundant computation by sharing results [27, 28].

2.4 Haskell for Language Implementation

Haskell's functional nature makes it ideal for implementing interpreters. Hudak [5] highlights Haskell's monadic constructs for managing state and parsing, which are critical for the development of the SOS interpreter. Furthermore, Hutton [6] demonstrates how Haskell can be used

to build parsers and interpreters for domain-specific languages, providing practical guidance for this project.

2.5 Parsing Techniques

Aho et al. [7] detail parsing techniques, including recursive descent parsing, which is suitable for L due to its simplicity and alignment with Haskell's functional style. The Parsec library in Haskell [8] offers robust tools to build parsers, which was considered for use in this project.

3 Requirements

This section outlines the functional and nonfunctional requirements of the project.

3.1 Functional Requirements

- **SOS Definition**: A formal SOS specification for L, covering syntax and semantics for basic constructs (e.g. variables, arithmetic, conditionals).
- **Parser**: A Haskell-based parser to validate and process L programs.
- **SOS-Interpreter**: An interpreter implementing the SOS rules to execute L programs.
- **User Interface**: A simple interface, command-line and graphical to input L programs and display evaluation state and results.

3.2 Non-Functional Requirements

- **Modularity**: The interpreter and parser should be modular to facilitate future extensions.
- **Reliability**: The interpreter should accurately reflect the SOS-defined behavior.
- **Usability**: The interface should be intuitive, with clear output for program execution.

4 Syntax & Semantics of *L*

4.1 Overview

*L*is designed as a simple functional language based on lambda calculus, as discussed above. It is a language for writing very simple mathematical expressions. Its purpose is to provide a clear and manageable target for demonstrating the application of SOS and it's accompanying interpreter implementation. The concrete syntax of L is defined using the following Backus-Naur Form (BNF) grammar 1. This grammar specifies the textual representation of valid L programs.

4.2 SOS Definition for L

A formal SOS for *L*, specifying syntax (e.g., BNF notation) and transition rules for execution. This section defines its syntax and operational semantics.

4.2.1 BNF Grammar for L

The syntax of L is defined using the Backus-Naur form (BNF) for expressions in infix notation:

Listing 1: BNF Grammar for L

```
expr ::= num
       | bool
       | var
       | '\' var+ '->' expr              // Lambda abstraction
       | expr expr                       // Application (left-associative
          )
       | expr '+' expr
       | expr '-' expr
       | expr '*' expr
       | expr '==' expr
       | 'let' var '=' expr 'in' expr
       | 'if' expr 'then' expr 'else' expr
       | '[' expr_list ']'
       | 'cons' expr expr
       | 'head' expr
       | 'tail' expr
       | 'isEmpty' expr
       | '(' expr ')'

expr_list ::= expr
            | expr ',' expr_list
            | /* empty */

num  ::= /* integer literal */
bool ::= 'True' | 'False'
var  ::= /* identifier */
```

This grammar snippet illustrates how expressions are structured, from basic atomic units to complex compositions involving arithmetic operations, function definitions (\, λ), function application, `let` bindings, conditionals, and list operations. Operator precedence (e.g., `*` before `+`) and associativity (e.g., left-associativity for `+`, `-`, `*`, and application) are implicitly handled by the tiered structure of the parser rules derived from this grammar.

Examples of valid L expressions include:

- `3` (a number)
- `3 + 4` (addition, evaluates to 7)
- `(2 + 3) * 4` (multiplication of an addition, evaluates to 20)
- `(3 - 1) * 4` (multiplication of an subtraction, evaluates to 8)
- `(2 + 2) / 4` (division of an addition, evaluates to 1)
- `3 + 4` (from L, evaluates to 7)
- `let x = 5 in x + 3` (binds `x` to 5, evaluates to 8)
- `(λ x.x + 2)(3)` (applies a function adding 2 to 3, evaluates to 5)
- `if 5 > 3 then 1 else 0` (evaluates to 1)

4.2.2 Operational Semantics of L

The operational semantics of L uses small-step SOS with a state consisting of a pair $\langle e, \sigma \rangle$, where e is an expression and σ is an environment mapping variables to values (numbers or closures). The reduction relation is $\langle e, \sigma \rangle \to \langle e', \sigma' \rangle$. Values include numbers (n) and lambda expressions ($\lambda x.e$).

The reduction rules include the following.

$$\frac{\langle e_1, \sigma \rangle \to \langle e_1', \sigma \rangle}{\langle e_1 + e_2, \sigma \rangle \to \langle e_1' + e_2, \sigma \rangle} \quad \text{(E-Add1)} \tag{1}$$

$$\frac{\langle e_2, \sigma \rangle \to \langle e_2', \sigma \rangle}{\langle n_1 + e_2, \sigma \rangle \to \langle n_1 + e_2', \sigma \rangle} \quad \text{(E-Add2)} \tag{2}$$

$$\langle n_1 + n_2, \sigma \rangle \to \langle n_3, \sigma \rangle \quad \text{where } n_3 = n_1 + n_2 \quad \text{(E-AddCompute)} \tag{3}$$

Similar rules apply for subtraction, multiplication, and division. For variables:

$$\langle x, \sigma \rangle \to \langle v, \sigma \rangle \quad \text{if } \sigma(x) = v \quad \text{(E-Var)} \tag{4}$$

For let bindings (extending the environment):

$$\frac{\langle e_1, \sigma \rangle \to \langle e_1', \sigma \rangle}{\langle \text{let } x = e_1 \text{ in } e_2, \sigma \rangle \to \langle \text{let } x = e_1' \text{ in } e_2, \sigma \rangle} \quad \text{(E-Let1)} \tag{5}$$

$$\langle \text{let } x = v \text{ in } e_2, \sigma \rangle \to \langle e_2, \sigma[x \mapsto v] \rangle \quad \text{(E-Let2)} \tag{6}$$

For conditional expressions:

$$\frac{\langle e_1, \sigma \rangle \to \langle e_1', \sigma \rangle}{\langle \text{if } e_1 \text{ then } e_2 \text{ else } e_3, \sigma \rangle \to \langle \text{if } e_1' \text{ then } e_2 \text{ else } e_3, \sigma \rangle} \quad \text{(E-If)} \tag{7}$$

$$\langle \text{if true then } e_2 \text{ else } e_3, \sigma \rangle \to \langle e_2, \sigma \rangle \quad \text{(E-IfTrue)} \tag{8}$$

$$\langle \text{if false then } e_2 \text{ else } e_3, \sigma \rangle \to \langle e_3, \sigma \rangle \quad \text{(E-IfFalse)} \tag{9}$$

For List operations:

$$\frac{\langle e_1, \sigma \rangle \to \langle e_1', \sigma \rangle}{\langle \text{cons}(e_1, e_2), \sigma \rangle \to \langle \text{cons}(e_1', e_2), \sigma \rangle} \quad \text{(E-Cons1)}$$

$$\frac{\langle e_2, \sigma \rangle \to \langle e_2', \sigma \rangle}{\langle \text{cons}(v_1, e_2), \sigma \rangle \to \langle \text{cons}(v_1, e_2'), \sigma \rangle} \quad \text{(E-Cons2)}$$

$$\frac{\langle e_l, \sigma\rangle \to \langle e_l', \sigma\rangle}{\langle \text{head}(e_l), \sigma\rangle \to \langle \text{head}(e_l'), \sigma\rangle} \quad \text{(E-HeadEval)}$$

$$\langle \text{head}(\text{cons}(v_h, lv_t)), \sigma\rangle \to \langle v_h, \sigma\rangle \quad \text{(E-HeadCompute)}$$

$$\langle \text{head}(\text{nil}), \sigma\rangle \to \langle \text{error}_{\text{head_on_nil}}, \sigma\rangle \quad \text{(E-HeadNilError)} \tag{10}$$

$$\frac{\langle e_l, \sigma\rangle \to \langle e_l', \sigma\rangle}{\langle \text{tail}(e_l), \sigma\rangle \to \langle \text{tail}(e_l'), \sigma\rangle} \quad \text{(E-TailEval)}$$

$$\langle \text{tail}(\text{cons}(v_h, lv_t)), \sigma\rangle \to \langle lv_t, \sigma\rangle \quad \text{(E-TailCompute)}$$

$$\langle \text{tail}(\text{nil}), \sigma\rangle \to \langle \text{error}_{\text{tail_on_nil}}, \sigma\rangle \quad \text{(E-TailNilError)}$$

$$\frac{\langle e_l, \sigma\rangle \to \langle e_l', \sigma\rangle}{\langle \text{isEmpty}(e_l), \sigma\rangle \to \langle \text{isEmpty}(e_l'), \sigma\rangle} \quad \text{(E-IsEmptyEval)}$$

$$\langle \text{isEmpty}(\text{nil}), \sigma\rangle \to \langle \text{true}, \sigma\rangle \quad \text{(E-IsEmptyNil)}$$

$$\langle \text{isEmpty}(\text{cons}(v_h, lv_t)), \sigma\rangle \to \langle \text{false}, \sigma\rangle \quad \text{(E-IsEmptyCons)}$$

For function application (Dynamic Scope):

$$\frac{\langle e_1, \sigma\rangle \to \langle e_1', \sigma\rangle}{\langle e_1(e_2), \sigma\rangle \to \langle e_1'(e_2), \sigma\rangle} \quad \text{(E-App1)} \tag{11}$$

$$\frac{\langle e_2, \sigma\rangle \to \langle e_2', \sigma\rangle}{\langle v(e_2), \sigma\rangle \to \langle v(e_2'), \sigma\rangle} \quad \text{(E-App2)} \tag{12}$$

$$\langle (\lambda x.e)(v), \sigma\rangle \to \langle e, \sigma[x \mapsto v]\rangle \quad \text{(E-AppLambda)} \tag{13}$$

An expression is reduced step by step until it becomes a value (a number). For example, `(2 + 3) * 4` reduces as follows:

$$\begin{aligned}(2+3)*4 &\to 5*4\\ &\to 20\end{aligned}$$

5 Design & Implementation

Understanding the implementation requires clarity on several fundamental programming language concepts:

- **Syntax vs. Semantics**: Syntax defines the structure and form of valid programs (what they look like), while semantics defines their meaning and behavior (what they do) [20]. SOS is concerned with defining semantics.
- **Concrete vs. Abstract Syntax**: Concrete syntax refers to the exact sequence of characters or tokens that a programmer writes, including details like parentheses, keywords, and operator precedence rules. Abstract Syntax focuses on the essential hierarchical structure of the program, typically represented as a tree, abstracting away superficial syntactic details [18]. SOS rules operate on the abstract syntax.
- **Abstract Syntax Trees (ASTs)**: ASTs are data structures that are used to represent programs internally within compilers and interpreters after parsing. They capture the structure defined by the abstract syntax, making them suitable for semantic analysis, transformation, and evaluation [18]. The SOS rules are defined based on the structure of these ASTs.

5.1 Interpreter Architecture Overview

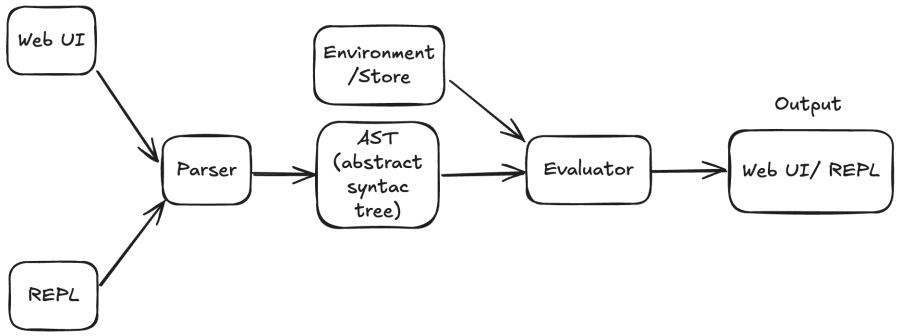


Figure 1: Architecture of the L interpreter.

The interpreter follows a standard pipeline architecture:

- **Parser**: Takes the raw source code of an L program (as a string) and transforms it into an Abstract Syntax Tree (AST) representation. If parsing fails, an error is reported.
- **Evaluator**: Takes the AST and an initial store, and applies the small-step operational semantics rules iteratively using a core step function.
- **Execution Loop**: Manages the repeated application of the step function, starting from the initial configuration ⟨Program AST, Initial Store/Environment⟩ until a terminal configuration (final store) or an error state is reached.
- **User Interface (UI)**: Provides a means for the user to input L programs, initiate execution, and observe the final results (e.g., the final store content).

The implementation is modularized, with distinct Haskell modules planned for parsing (`Parser.hs`), AST definitions (`AST.hs`), the core interpreter logic (`Evaluator.hs`), and the main program entry point and UI (`Main.hs`). The abstract syntax of language L is represented in Haskell using Algebraic Data Types (ADTs), mirroring the structure defined in the BNF grammar 1 and serving as the input to the interpreter's evaluation logic.

5.2 Viability of Haskell as an implementation language

Functional programming languages such as Haskell are widely recognized as highly suitable for implementing compilers and interpreters, especially those based on formal semantic specifications such as SOS. Key features contributing to this suitability include:

- **Algebraic Data Types (ADTs)**: ADTs provide a concise way to represent the recursive structure of Abstract Syntax Trees (ASTs).
- **Pattern Matching**: Haskell's powerful pattern matching allows for elegant dispatching based on the structure of the AST, directly mirroring the selection of SOS rules based on syntactic forms.
- **Recursion**: Recursive functions are the natural way to traverse and evaluate the recursive structures of ASTs, aligning well with the inductive nature of semantic definitions.
- **Strong Typing**: Haskell's static type system helps catch errors at compile time, improving the robustness of the interpreter implementation.
- **Higher-Order Functions and Laziness**: These features facilitate modular design and can be advantageous for implementing certain semantic features or evaluation strategies.

This close correspondence between the structures used in formal semantics and the core features of Haskell makes mapping and translating the semantics directly to working program remarkably simple and declarative.

5.3 Ast Implementation

The interpreter uses Haskell for its type safety and functional purity. These ADTs correspond directly to the syntactic categories of L. The `Expr` type recursively defines expressions, and the Stmt type defines the structure of statements. The Store is represented as `Map` mapping variable names (Strings) to their Values. This representation provides a type-safe and structured way for interpreter functions to operate on L programs

5.3.1 Abstract Syntax Tree for L

The Abstract Syntax Tree (AST) is an internal, tree-like representation of the parsed program, abstracting away from concrete syntactic details like parentheses or specific keyword spellings. The `Expr` data type in `Ast.hs` defines the structure of these AST nodes:

Listing 2: AST definition for L

```
module Ast where
-- ------------------------------------------------------------
-- Abstract Syntax Tree (AST)
-- ------------------------------------------------------------

data Expr
  = Var String            -- Variable: x
  | Lam String Expr       -- Lambda Abstraction: \x -> e
  | App Expr Expr         -- Application: e1 e2
  | Let String Expr Expr  -- Let Binding: let x = e1 in e2
  | Num Int               -- Integer Literal: 5
  | Add Expr Expr         -- Addition: e1 + e2
  | Sub Expr Expr         -- Subtraction: e1 - e2
  | Mul Expr Expr         -- Multiplication: e1 * e2
  | Eq Expr Expr          -- Equality Check: e1 == e2
  | List [Expr]           -- List Literal: [e1, e2]
```

```
  | Cons Expr Expr        -- List Constructor: cons e1 e2
  | Head Expr             -- List Head: head e
  | Tail Expr             -- List Tail: tail e
  | IsEmpty Expr          -- Check if list is empty: isEmpty e
  | BoolLit Bool          -- Boolean Literal: True, False
  | IfThenElse Expr Expr Expr -- Conditional: if cond then expr1 else
      expr2
  deriving (Show, Eq)
```

Each constructor in `Expr` corresponds to a distinct syntactic form in the language. For instance, `Lam String Expr` represents a function with a single parameter (the `String`) and a body (the `Expr`). Multiple parameters in the concrete syntax (e.g., `\x y -> ...`) are parsed into nested `Lam` nodes (e.g., `Lam "x" (Lam "y" (...))`) to naturally support currying. `App Expr Expr` represents the application of one expression (the function) to another (the argument).

5.4 Data types & Values

During evaluation, `Expr` AST nodes are transformed into `Value` types, which represent the runtime results of computations. The `Value` data type is defined in `Value.hs`:

Listing 3: Runtime Value Representation in `Value.hs`

```
-- From Value.hs
data Value
  = VNum Int
  | VBool Bool
  | VList [Value]
  | VClosure String Expr Env  -- Parameter name, function body (Expr),
      captured lexical environment (Env)
  | VPrim String (Value -> Either String Value) -- Name and Haskell
      implementation of a built-in primitive
  deriving (Eq, Show)
```

The `VNum`, `VBool`, and `VList` constructors represent simple atomic values. The most significant constructor for functional capabilities is `VClosure String Expr Env`, which represents a first-class function value (a closure). This closure packages:

- the function's formal parameter name (`String`),
- the function's body as an unevaluated `Expr`, and
- the lexical environment (`Env`) that was active when the function was defined.

This captured environment is crucial for implementing lexical scoping. The `VPrim` constructor is used for built-in functions provided by the interpreter itself.

5.5 Parser Implementation

Before interpretation, the source code string must be transformed into its AST representation. While this can be done manually, parser combinator libraries like Parsec [14] or its modern successor Megaparsec [15] are commonly used in Haskell [16] These libraries allow parsers to be defined as compositions of smaller parsers, often closely mirroring the structure of the language's BNF grammar. They handle complexities such as backtracking, error reporting, and consuming input, allowing the implementer to focus on defining the parsing logic for each language construct. The process typically involves defining a lexer to handle whitespace, comments, and basic tokens (identifiers, keywords, operators, literals), and then a parser that uses these tokens to build the AST. The abstraction provided by these libraries significantly simplifies the parsing phase, separating it cleanly from the semantic interpretation phase [16].

5.5.1 Parser Design (`Parser.hs`)

The parser, implemented in `Parser.hs`, is responsible for transforming a sequence of characters (the source code) into an `Expr` AST.

5.5.2 Choice of Parsing Technique

A monadic parser combinator approach is used. A custom parser type is defined:

```
newtype Parser a = Parser { parse ::  String -> Maybe (a, String) }
```

Instances for `Functor`, `Applicative`, `Monad`, `Alternative`, and `MonadPlus` are provided. This allows parsers to be constructed as small, composable functions.

5.5.3 Structure of the Parser Module

- **Basic Combinators:** Fundamental building blocks like `satisfy` (parses a character satisfying a predicate), `char` (parses a specific character), and `string` (parses a specific string).
- **Whitespace Handling:** `ws` consumes whitespace, and `token` wraps other parsers to consume surrounding whitespace, simplifying grammar rules.
- **Lexemes/Tokens:** Parsers for basic lexical units like `identifier` (for variable names, checking against `reservedWords`) and `integer` (for numeric literals).
- **Expression Parsers:** A set of mutually recursive functions handle various expression types, organized by precedence:
  - `parseAtom`: Parses tightly binding expressions like literals (`parseNum`, `parseBool`), variables (`parseVar`), parenthesized expressions (`parseParen`), list literals (`parseList`), lambda abstractions (`parseLambda`), and list operations (`parseConsKeyword`, etc.).
  - `parseMul`, `parseAddSub`, `parseEq`: These use the `chainl1` combinator to enforce left-associativity and operator precedence:
    - `parseMul` for `*`
    - `parseAddSub` for `+` and `-`
    - `parseEq` for `==`
  - `parseIf`, `parseLet`: Parsers for `if`- and `let`-based constructs.
  - `parseExpr`: The top-level parser. It attempts to parse expressions in this order: `if`, `let`, `lambda` (via `parseLambda`), then falls back to operator precedence parsing (starting with `parseEq`).

5.5.4 Parsing Lambdas (`parseLambda`)

The `parseLambda` function parses lambda abstractions like `\x -> x + 1` or `x y -> x + y`. The steps are:

1. Expect a `\` or character.
2. Parse one or more identifiers for parameter names using `some identifier`.
3. Expect the `->` token.
4. Recursively parse the function body using `parseExpr`.
5. Use `foldr Lam body vars` to support currying. For example, `\x y -> E` becomes `Lam "x" (Lam "y" E)`.

Ensuring Correct Precedence

Operator precedence and associativity are handled by the stratified structure of the expression parsing functions and the use of `chainl1`. The binding hierarchy is as follows:

- `parseAtom`: binds tightest (literals, variables, parenthesis, lambdas)
- `parseApp`: next tightest (function application)
- `parseMul`: for `*`
- `parseAddSub`: for `+` and `-`
- `parseEq`: for `==`
- `parseIf`, `parseLet`, `parseLambda` (when used top-level): loosest binding or handled first via `parseExpr`

The full precedence chain is:

`parseExpr` → `parseIf / parseLet / parseLambda / parseEq` → `parseAddSub` → `parseMul` → `parseApp` → `parseAtom`

Listing 4: Type definition for custom parser

```
newtype Parser a = Parser {parse :: String -> Maybe (a, String)}

instance Functor Parser where
  fmap f (Parser p) = Parser $ \s -> do
    (a, s') <- p s
    return (f a, s')

instance Applicative Parser where
  pure a = Parser $ \s -> Just (a, s)
  (<*>) = ap -- Use Control.Monad.ap for Applicative (<*>)

instance Monad Parser where
  return = pure
  Parser p >>= f = Parser $ \s -> do
    (a, s') <- p s
    parse (f a) s'

instance Alternative Parser where
  empty = Parser $ \_ -> Nothing
  Parser p1 <|> Parser p2 = Parser $ \s -> p1 s <|> p2 s

instance MonadPlus Parser where
      mzero = empty
      mplus = (<|>)
```

5.6 Interpreter Implementation

The SOS-interpreter will execute L programs according to the defined transition rules. The core of the interpreter is the eval function (shown in listing 6), which implements the small-step transition relation (→) defined by our SOS rules. This function takes the current configuration (an expression and environment) and returns the configuration after one step of execution. An example of the environment is represented in listing 5

5.6.1 Evaluator Design (`Evaluator.hs`)

The evaluator, implemented in `Evaluator.hs`, is the heart of the interpreter. It takes an `Expr` (AST node) and an `Env` (environment) and computes a `Value`.

5.6.2 Main Evaluation Function Signature

The core evaluation function is:

```
eval ::  Env -> Expr -> Either String (Value, TraceLog)
```

- `Env`: The current lexical environment, mapping variable names (`String`) to their `Value`.
- `Expr`: The AST node to evaluate.
- `Either String (Value, TraceLog)`: The result is either an error message (`Left String`) or a pair of the computed value and an evaluation trace log (`Right (Value, TraceLog)`). The `Either` monad is used for principled error handling.

5.6.3 Environment Representation (`Env`)

As defined in `Value.hs`, the environment is represented as:

```
type Env = Map.Map String Value
```

This maps variable names to their current values in scope. `Data.Map` is used for efficient lookup and extension. The initial environment, `initialEnv`, is typically an empty map, possibly pre-populated with built-in primitives.

5.6.4 Evaluation of Lambda Abstractions (`Lam var body`)

When a `Lam var body` expression is encountered, the body is **not** evaluated immediately. Instead, the evaluator constructs a closure:

```
VClosure var body env
```

- `var` is the parameter name.
- `body` is the unevaluated function body (`Expr`).
- `env` is the current lexical environment at the time the lambda is evaluated.

This environment capture is what enables lexical scoping. The result of evaluating a lambda is:

```
Right (VClosure var body env, trace)
```

5.6.5 Evaluation of Function Application (`App fun arg`)

This is the most complex and crucial part of functional evaluation:

1. **Evaluate the function expression:**

   ```
   (funVal, funLog) <- eval env fun
   ```

   The result should be a function, typically `VClosure` or `VPrim`.

2. **Evaluate the argument expression:**

   `(argVal, argLog) <- eval env arg`

   This yields the value to pass to the function.

3. **Apply the function:**

   - **If** `funVal` is `VClosure cParam cBody cEnv`:
     - Extend the captured environment: `extendedEnv = Map.insert cParam argVal cEnv`
     - Evaluate the body: `(resVal, bodyLog) <- eval extendedEnv cBody`
     - Return: `Right (resVal, combinedTraceLog)`
   - **If** `funVal` is `VPrim name primFn`:

     `case primFn argVal of Left err -> Left err; Right resPrim -> Right (resPrim, combinedTraceLog)`
   - **Otherwise:** If `funVal` is not a function (e.g., `VNum`), return a type error:

     `Left "Attempted to apply non-function"`

5.6.6 Scope Management for Let Expressions and Function Parameters

`Let var boundExpr bodyExpr`:

1. Evaluate `boundExpr` in the current environment to get `boundVal`.
2. Create a new environment: `env' = Map.insert var boundVal env`
3. Evaluate `bodyExpr` in `env'`.

The binding `var = boundVal` is only visible within `bodyExpr` and does not affect the evaluation of `boundExpr` unless recursive constructs are added explicitly.

5.6.7 Function Parameters:

As described in application evaluation, when a `VClosure` is applied:

- The parameter is bound to the argument in a new environment derived from the closure's captured `env`.
- This binding is local to the evaluation of the function body and does not alter the global or call-site environment.

Listing 5: Initial Environment

```
type Env = Map.Map String Value

initialEnv :: Env
initialEnv = Map.empty
```

Listing 6: Full intepreter for complete L and Lx

```
-- Main eval function
eval :: Env -> Expr -> Either String (Value, TraceLog)
eval env e = case e of
  Num n ->
    let v = VNum n
```

```
6           trace = ["Eval␣Num␣" ++ show n ++ "␣->␣" ++ show v]
7       in Right (v, trace)
8   BoolLit b ->
9     let v = VBool b
10          trace = ["Eval␣BoolLit␣" ++ show b ++ "␣->␣" ++ show v]
11      in Right (v, trace)
12  Var x ->
13    let entryLog = "Eval␣Var␣" ++ x
14     in case Map.lookup x env of
15          Just v -> Right (v, [entryLog ++ "␣->␣" ++ show v])
16          Nothing -> Left ("Undefined␣variable:␣" ++ x)
17  Lam var body ->
18    let v = VClosure var body env -- Crucially captures current env
19        trace = ["Eval␣Lam␣" ++ var ++ "␣->␣" ++ show v]
20     in Right (v, trace)
21  App fun arg -> do
22    let entryLog = "Eval␣App␣(" ++ show fun ++ ")␣(" ++ show arg ++ ")"
23    (funVal, funLog) <- eval env fun
24    (argVal, argLog) <- eval env arg
25    let applyLog = "␣Apply␣" ++ show funVal ++ "␣to␣" ++ show argVal
26    case funVal of
27      VClosure var body closureEnv -> do
28        (resVal, bodyLog) <- eval (Map.insert var argVal closureEnv)
              body
29        Right (resVal, [entryLog] ++ indentLog funLog ++ indentLog
              argLog ++ [applyLog] ++ indentLog bodyLog ++ ["␣␣->␣" ++
              show resVal])
30      VPrim name primFn ->
31        case primFn argVal of -- Primitive itself doesn't return a log
              directly here
32          Left err -> Left err
33          Right resPrim -> Right (resPrim, [entryLog] ++ indentLog
              funLog ++ indentLog argLog ++ [applyLog, "␣␣Primitive␣" ++
               name ++ "␣->␣" ++ show resPrim])
34      _ -> Left ("Type␣Error:␣Attempted␣to␣apply␣non-function:␣" ++
           show funVal)
35  Let var boundExpr bodyExpr -> do
36    let entryLog = "Eval␣Let␣" ++ var ++ "␣=␣" ++ show boundExpr ++ "␣
        in␣..."
37    (boundVal, boundLog) <- eval env boundExpr
38    let env' = Map.insert var boundVal env
39    let letStepLog = "␣␣With␣" ++ var ++ "␣=␣" ++ show boundVal
40    (bodyVal, bodyLog) <- eval env' bodyExpr
41    Right (bodyVal, [entryLog] ++ indentLog boundLog ++ [letStepLog] ++
        indentLog bodyLog ++ ["␣␣->␣" ++ show bodyVal])
42  IfThenElse cond thn els -> do
43    let entryLog = "Eval␣If␣" ++ show cond ++ "␣then␣..."
44    (condVal, condLog) <- eval env cond
45    let stepCond = "␣Condition␣" ++ show cond ++ "␣evaluated␣to␣" ++
        show condVal
46    case condVal of
47      VBool True -> do
48        (thnVal, thnLog) <- eval env thn
49        Right (thnVal, [entryLog] ++ indentLog condLog ++ [stepCond, "␣
              ␣Then␣branch␣taken"] ++ indentLog thnLog ++ ["␣␣->␣" ++ show
               thnVal])
50      VBool False -> do
51        (elsVal, elsLog) <- eval env els
```

```
52          Right (elsVal, [entryLog] ++ indentLog condLog ++ [stepCond, "␣
               ␣Else␣branch␣taken"] ++ indentLog elsLog ++ ["␣␣␣->␣" ++ show
                elsVal])
53        _ -> Left "Type␣Error:␣'if'␣condition␣must␣evaluate␣to␣a␣Boolean.
             "
54    Add e1 e2 -> evalBinaryOp e1 e2 env "+" (\n1 n2 -> VNum (n1 + n2))
         isVNum "numbers"
55    Sub e1 e2 -> evalBinaryOp e1 e2 env "-" (\n1 n2 -> VNum (n1 - n2))
         isVNum "numbers"
56    Mul e1 e2 -> evalBinaryOp e1 e2 env "*" (\n1 n2 -> VNum (n1 * n2))
         isVNum "numbers"
57    Eq e1 e2 ->
58      evalBinaryOp
59        e1
60        e2
61        env
62        "=="
63        (\_n1 _n2 -> error "Arithmetic␣function␣should␣not␣be␣called␣for␣
             Eq") -- Dummy fn
64        (\_val -> True) -- Type check for Eq operands: any value is fine
65        "comparable␣values"
66    List exprs -> do
67      let entryLog = "Eval␣List␣[" ++ show (length exprs) ++ "␣elements]"
68      -- Evaluate all elements and collect values and logs
69      resultsAndLogs <- mapM (eval env) exprs
70      let (vals, logsNested) = unzip resultsAndLogs
71      let resVal = VList vals
72      Right (resVal, [entryLog] ++ concatMap indentLog logsNested ++ ["␣␣
           ->␣" ++ show resVal])
73    Cons e1 e2 -> do
74      let entryLog = "Eval␣Cons␣(" ++ show e1 ++ ")␣(" ++ show e2 ++ ")"
75      (v1, log1) <- eval env e1
76      (v2, log2) <- eval env e2
77      let consStepLog = "␣Consing␣" ++ show v1 ++ "␣with␣" ++ show v2
78      case v2 of
79        VList vs ->
80          let resVal = VList (v1 : vs)
81           in Right (resVal, [entryLog] ++ indentLog log1 ++ indentLog
                log2 ++ [consStepLog, "␣␣␣->␣" ++ show resVal])
82        _ -> Left "Type␣Error:␣Second␣argument␣to␣'cons'␣must␣be␣a␣list."
83    Head e -> do
84      let entryLog = "Eval␣Head␣(" ++ show e ++ ")"
85      (v, logSub) <- eval env e
86      let headStepLog = "␣Head␣of␣" ++ show v
87      case v of
88        VList (x : _) -> Right (x, [entryLog] ++ indentLog logSub ++ [
             headStepLog, "␣␣␣->␣" ++ show x])
89        VList [] -> Left "Runtime␣Error:␣Cannot␣take␣head␣of␣an␣empty␣
             list."
90        _ -> Left "Type␣Error:␣'head'␣requires␣a␣list␣argument."
91    Tail e -> do
92      let entryLog = "Eval␣Tail␣(" ++ show e ++ ")"
93      (v, logSub) <- eval env e
94      let tailStepLog = "␣Tail␣of␣" ++ show v
95      case v of
96        VList (_ : xs) ->
97          let resVal = VList xs
98           in Right (resVal, [entryLog] ++ indentLog logSub ++ [
```

```
            tailStepLog, "␣␣->␣" ++ show resVal])
      VList [] -> Left "Runtime␣Error:␣Cannot␣take␣tail␣of␣an␣empty␣
          list."
      _ -> Left "Type␣Error:␣'tail'␣requires␣a␣list␣argument."
  IsEmpty e -> do
    let entryLog = "Eval␣IsEmpty␣(" ++ show e ++ ")"
    (v, logSub) <- eval env e
    let isEmptyStepLog = "␣IsEmpty␣on␣" ++ show v
    case v of
      VList [] -> Right (VBool True, [entryLog] ++ indentLog logSub ++
          [isEmptyStepLog, "␣␣->␣True"])
      VList _ -> Right (VBool False, [entryLog] ++ indentLog logSub ++
          [isEmptyStepLog, "␣␣->␣False"])
      _ -> Left "Type␣Error:␣'isEmpty'␣requires␣a␣list␣argument."
```

This implementation structure directly reflects the SOS definition. Each clause in the step function corresponds to one or more SOS rules, selected via pattern matching on the *Expr* constructor. We also keep track of every evaluation step in a *TraceLog* data structure which is of the type: *[String]*, which essentially stores a string representation of the evaluation step.

5.7 User Interface

A command-line interface and web-based user interface will allow users to input L programs and view evaluation results. The web interface is a ReactJS and TypeScript web application that consists of 3 main sections, navigation, which includes buttons that:

- **Evaluate**: This sends the l program to the server that runs the interpreter.
- **Save**: This allows the user to save the current l-program as a snippet.
- **Snippets**: This allows the user to view, select and prefill the text editor with contents of the selected snippet.
- **Clear**: This clears the current state of the editor.

The next section of is a grid of the text editor where the user inputs l programs and the right-hand side of the interface that shows the output of the evaluated expression. The output section is able to show the output, the evaluation steps, the current environment and the parsed AST.

5.7.1 Evaluating L programs via the REPL

The image below 2 shows how a user would interact with the *L* interpreter through the terminal in a Read Evaluate Print Loop (REPL) environment. Enabled by some commands, the user can exit this environment with the `:quit` command. The `:env` command reveals the current state of the environment, as an association list of bindings and bound values/expressions.

```
L Programming language
Define: name = expression (recursion supported)
Evaluate: expression (e.g., if x == 0 then 1 else x * fact (x-1))
Commands: :quit, :env
L-Repl> double = \x → x * 2
Evaluating potentially recursive definition for 'double'...
Defined (rec): double
L-Repl> :env
fromList [("double",<closure: \x → ...>)]
L-Repl> fact = \x → if x == 0 then 1 else x * fact (x - 1)
Evaluating potentially recursive definition for 'fact'...
Defined (rec): fact
L-Repl> fact double 2
Error: Type Error: Cannot compare <closure: \x → ...> and 0 with ==
L-Repl> fact 5
120
L-Repl> fact 1
1
L-Repl> fact 0
1
L-Repl> fact 1000
0
L-Repl> fact -1
Error: Arguments must be numbers for -
L-Repl> let x = 5 in x * double 5
50
L-Repl> :env
fromList [("double",<closure: \x → ...>),("fact",<closure: \x → ...>)]
L-Repl> :quit
Goodbye!
```

Figure 2: L REPL showing evaluated results L.

5.7.2 Evaluating L programs via Web interface

The images below shows an example L program that defines a factorial binding and applies it to the number 5. Here we show the evaluation steps (call by value) as the interpreter evaluates the factorial function applied to 5.

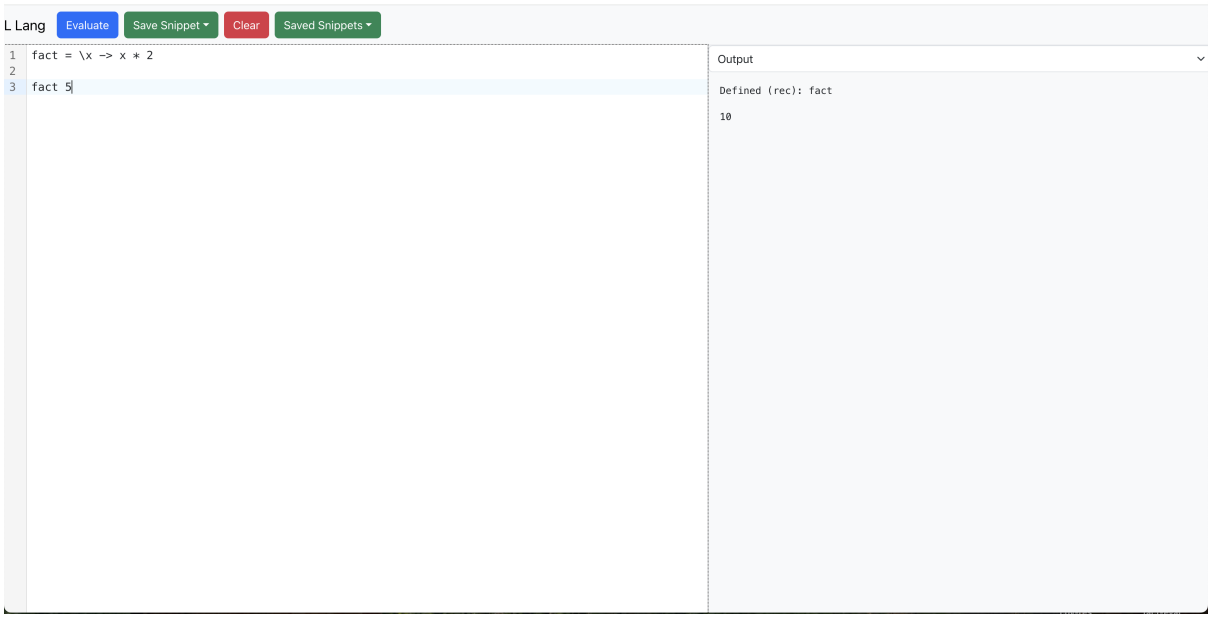


Figure 3: Evaluated output of definition and application of factorial

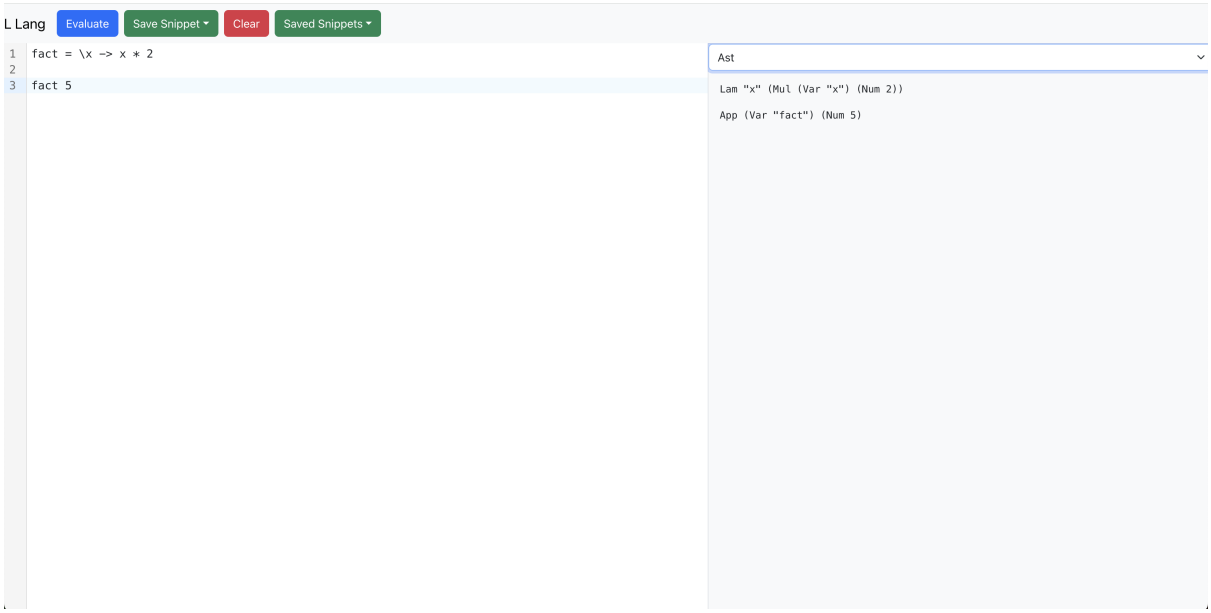


Figure 4: Parsed AST of factorial function and application

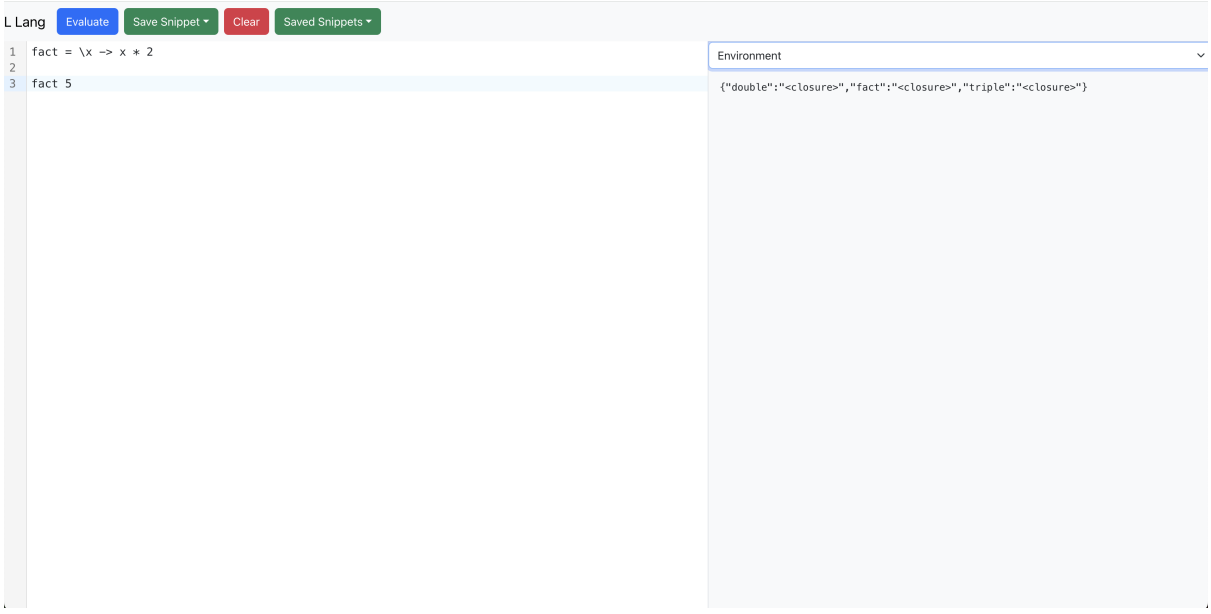


Figure 5: Current state of environment after definition and application of factorial

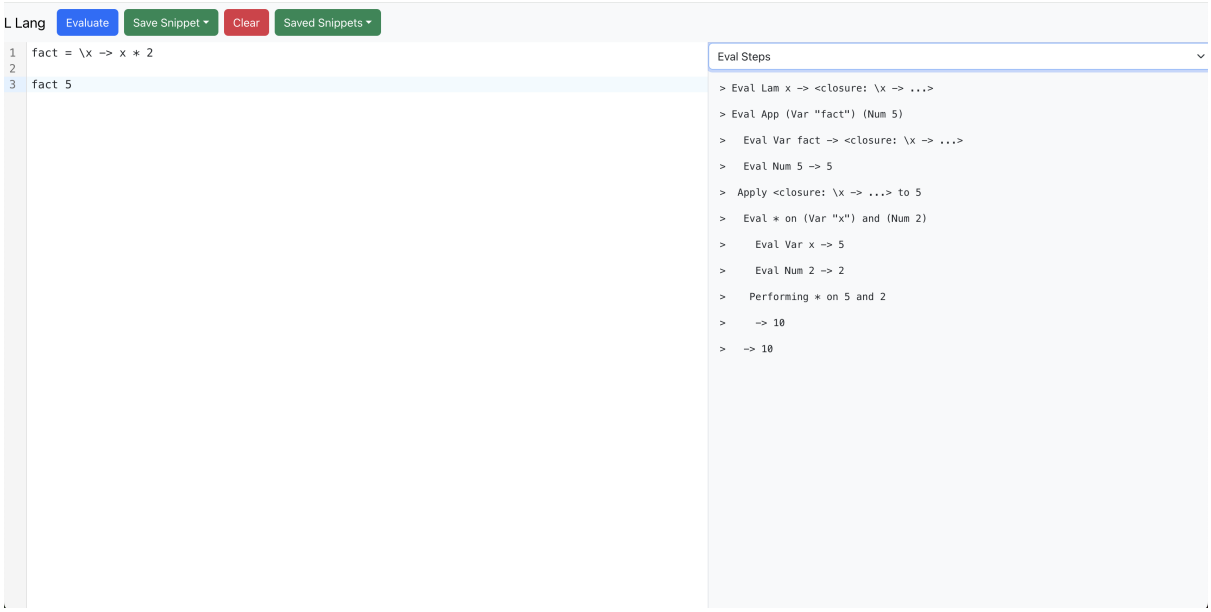


Figure 6: Evaluation steps of definition and application of factorial

6 Implementation Details

The project is organized into several Haskell modules, each with a distinct responsibility, promoting a clear separation of concerns and maintainability.

6.1 Module Responsibilities

6.1.1 `Ast.hs` (Abstract Syntax Tree)

This module defines the `Expr` algebraic data type. It is the foundational data structure representing the parsed source code. All other modules that deal with the program's structure (like `Parser.hs` and `Evaluator.hs`) depend on `Ast.hs`.

6.1.2 `Value.hs` (Runtime Values and Environment)

This module defines the `Value` algebraic data type, which represents the possible results of evaluating expressions (e.g., numbers, booleans, lists, and crucially, `VClosure` for functions). It also defines the `Env` type alias (`Map.Map String Value`) for the environment that maps variable names to values. `Evaluator.hs` heavily relies on `Value.hs` to construct and manipulate these runtime entities.

6.1.3 `Parser.hs` (Source Code Parsing)

This module is responsible for converting input strings (source code) into an `Expr` AST. It defines the `Parser` type and uses parser combinators to implement the language's grammar. It depends on `Ast.hs` (to construct `Expr` values) but is largely independent of the evaluation logic. Its output (`Expr`) is the primary input to the evaluator.

6.1.4 `Evaluator.hs` (Expression Evaluation)

This is the core of the interpreter. It contains the `eval` function that takes an `Expr` from the parser and an `Env`, and recursively computes a `Value`. It depends on `Ast.hs` (to deconstruct

`Expr` nodes), `Value.hs` (to work with `Value`s and `Env`, and to create `VClosure`s), and `Parser.hs` (implicitly, as it consumes its output; also, `Evaluator.hs` imports `Parser` but doesn't directly use it in the `eval` logic, which is good for separation).

6.1.5 `Main.hs` (REPL and Web Interface)

This module provides the user interface(s) to the interpreter. It handles user input, calls `Parser.hs` to parse the input into an `Expr`, and then calls `Evaluator.hs` to evaluate the `Expr`. It manages the top-level environment for the REPL, allowing definitions to persist across multiple lines of input. It demonstrates the practical integration of the parser and evaluator.

6.2 Flow of Data and Interaction

6.2.1 Backend data flow

1. The user provides an input string (e.g., in the REPL).
2. `Main.hs` passes this string to `Parser.parse parseExpr inputString`.
3. `Parser.hs` attempts to convert the string into an `Expr` AST, defined in `Ast.hs`.
4. If parsing is successful, `Main.hs` takes the resulting `Expr` and the current top-level `Env` (from `Value.hs`) and calls `Evaluator.eval env expr`.
5. `Evaluator.hs` traverses the `Expr`, using and updating the `Env` as needed (e.g., for `let` bindings or function calls). During this process, it creates and manipulates `Value`s (defined in `Value.hs`), including `VClosure`s when `Lam` expressions are encountered.
6. `Evaluator.hs` returns either an error string or a `Value` (and a trace log) back to `Main.hs`.
7. `Main.hs` displays the result or error to the user and updates its top-level environment if a definition was made.

The `Env` type (mapping variable names to `Value`s) and the `Value` type (especially `VClosure`, which itself contains an `Env`) are central to the state and semantics of the evaluation process, enabling lexical scoping and first-class functions. The `Expr` type serves as the bridge between the textual representation and semantic interpretation.

6.2.2 Frontend Implementation and Data Flow (React)

The web interface for the L-Lang interpreter provides an interactive environment for users to write, evaluate, and manage code snippets. It is built as a Single-Page Application (SPA) using `React` with `TypeScript`, leveraging components from `React-Bootstrap` for styling and UI elements, and `CodeMirror` for the code editor.

Core Components and Structure

The frontend application is primarily structured around three main components:

6.2.3 App.tsx (Main Application Component)

Role: This is the root component that orchestrates the entire frontend application. It manages the overall application state, handles communication with the backend Haskell interpreter, and passes data and callbacks down to its child components.

State Management: `App.tsx` holds key pieces of state using React's `useState` hook:

- `code`: The current L-Lang code in the editor.
- `result`: An array of evaluation step results (output and AST) from the backend.
- `env`: The final environment state after evaluation.
- `traceLog`: The trace log of evaluation steps.
- `error`: Any error messages returned from the backend.
- `snippets`: An array of saved code snippets, persisted in `localStorage`.

Key Functions:

- `evaluateCode()`: Asynchronously sends the current code to the backend `/evaluate` endpoint and updates the state (`result`, `env`, `traceLog`, `error`) with the response.
- `handleEditorChange()`: Updates the `code` state as the user types in the CodeMirror editor.
- `handleSaveSnippet()`, `handleSelectSnippet()`, `handleAddNewSnippet()`: Manage the creation, selection, and persistence of code snippets in `localStorage` and update the `snippets` and `code` state accordingly.
- `handleClearState()`: Resets the editor and result displays.

Child Components: Renders `Navbar`, `CodeMirror`, and `Results` components.

Navbar.tsx (Navigation and Control Bar)

Role: Provides user controls for evaluating code, saving/loading snippets, and clearing the state.

Props: Receives functions (e.g., `evaluateCode`, `handleSaveSnippet`, `handleSelectSnippet`, `handleClearState`) and data (e.g., `snippets`, `currentSnippet`) from `App.tsx`.

Interaction: User actions (button clicks, dropdown selections) trigger the callback functions passed from `App.tsx`, such as calling `props.evaluateCode()` when the "Evaluate" button is clicked. It also manages local state for the snippet name input.

Results.tsx (Output Display Area)

Role: Displays the various outputs from the interpreter: the evaluated result, the Abstract Syntax Tree (AST) of evaluated expressions, the step-by-step evaluation trace, and the final environment.

Props: Receives `result`, `error`, `env`, and `traceLog` as props from `App.tsx`.

Functionality: Uses local state (`contentToShow`) and a `Form.Select` dropdown to allow users to switch between different views (Output, AST, Eval Steps, Environment). Each view is rendered by a specific content function.

The frontend follows a unidirectional data flow pattern common in React applications, primarily managed by the `App.tsx` component:

User Input

- The user types L-Lang code into the CodeMirror component. The `onChange` event triggers `handleEditorChange`, updating the `code` state in `App.tsx`.
- The user interacts with controls in `Navbar.tsx` (e.g., clicking "Evaluate", "Save Snippet", selecting a saved snippet, or clicking "Clear").

Action Handling

- Actions from `Navbar.tsx` call handler functions passed from `App.tsx`.
- For example, `evaluateCode()` sends the current code to the backend at `http://localhost:3000/evalua`

Backend Communication and State Update

- The Haskell backend (Scotty server) parses and evaluates the code and returns a JSON response with:
  - `finalEnvironment`
  - `steps` (output/AST pairs)
  - `finalError`
  - `traceLog`
- `App.tsx` updates its state using the `setSteps`, `setEnv`, `setTraceLog`, and `setError` functions.

Displaying Results

- The updated states are passed to `Results.tsx`, which renders the selected view (e.g., Output, AST).

Snippet Management

- Snippets are saved via `Navbar.tsx`, with logic in `App.tsx` updating both state and `localStorage`.
- Snippets are loaded using a `useEffect` hook when `App.tsx` mounts.

This component architecture and data flow create a reactive, user-friendly interface where actions propagate from the user, through `App.tsx`, and down into child components. Evaluation results are rendered clearly with support for persistent code snippets and detailed feedback from the interpreter backend.

6.3 Tools and Technologies Used

The development of this `L` interpreter and its associated components leveraged a range of modern tools and technologies, chosen for their suitability to the tasks of language implementation, web service development, and frontend user interface creation.

6.3.1 Interpreter Backend (Core Language Implementation)

6.3.2 Primary Language

Haskell was selected as the primary language for implementing the interpreter's core logic (`Parser.hs`, `Evaluator.hs`, `Ast.hs`, `Value.hs`). Its strong static typing, algebraic data types, pattern matching, and powerful support for functional programming paradigms made it an excellent choice for symbolic computation and language processing tasks.

6.3.3 Build System and Package Management

The project uses Haskell's standard build tools such as `Stack`, as indicated by the presence of configuration files like `package.yaml` or `stack.yaml`. These tools were used to manage dependencies, build the project, and run tests.

6.3.4 Key Haskell Libraries

- **base**: Fundamental Haskell libraries.
- **containers**: Provides efficient data structures like `Data.Map`, extensively used for environment mappings.
- **parsec** (or custom parser combinators): Although `Parser.hs` implements a custom `Parser` type and monadic combinators, the project lists `parsec` as a dependency, indicating possible use in earlier iterations or auxiliary functionality. The parsing logic in this report primarily reflects custom combinators inspired by libraries like `parsec`.
- **HUnit**: Used for writing unit tests to ensure the correctness of the interpreter's components.
- **hspec** and **QuickCheck**: Listed as dependencies, these libraries indicate a more comprehensive testing strategy beyond unit testing, possibly including property-based testing.
- **scotty**: A lightweight web framework used to build the backend API, facilitating communication between the frontend and the interpreter.
- **aeson**: Handles JSON serialization and deserialization between the Haskell backend and the React frontend.
- **text** and **vector**: Provide efficient text and array-like data manipulation, respectively.
- **wai-cors** and **wai-extra**: Middleware libraries used with `scotty` for handling CORS and other HTTP-related functionalities.

6.3.5 Frontend (Web Interface)

The web interface was developed using a modern JavaScript/TypeScript technology stack to allow users to interactively write and evaluate `L`-Lang programs.

- **React**: A popular JavaScript library used to build a dynamic and responsive single-page application (SPA).
- **TypeScript**: A typed superset of JavaScript that improves code quality, maintainability, and developer productivity.
- **Bootstrap**: A front-end CSS framework that provides pre-styled UI components and layout utilities for responsive design.

6.3.6 Version Control and Collaboration

- **Git**: The entire codebase was managed using Git for version control, supporting branching, commits, and merges.
- **GitLab**: A remote repository was used for hosting and tracking changes.

This combination of technologies enabled the creation of a robust and modular interpreter backend written in a functionally pure style, paired with an interactive and user-friendly web interface. The use of strong typing, parser combinators, closures, and a REPL interface highlights the suitability of Haskell for building educational programming language tools.

7 Testing and Evaluation

7.1 Testing Strategy

The primary method for ensuring the correctness and robustness of the interpreter was through systematic unit testing, facilitated by the HUnit testing framework for Haskell. This approach allowed for isolated testing of individual components, primarily the parser and the evaluator, and specific language features, including user-defined functions.

A key element of the testing strategy was the `evalTest :: String → Either String Value` helper function. This function encapsulates the parsing and evaluation pipeline: it takes a string representing a source code expression, attempts to parse it using `parseExpr` from `Parser.hs`, and if successful, evaluates the resulting AST using eval from `Evaluator.hs` with an initial empty environment. The function returns either the successfully computed Value (wrapped in `Right`) or an error message string (wrapped in `Left`).

Listing 7: Helper function for parsing and evaluating input expressions

```
-- Helper function to parse an input string and evaluate the resulting
    expression.
-- It discards the trace log, focusing on the resulting value or error.
evalTest :: String -> Either String Value
evalTest inputStr =
  case parse parseExpr inputStr of
    Nothing -> Left "TEST_ERROR: Parser failed to parse the input
       string."
    Just (expr, "") ->
      -- Successfully parsed and consumed all input
      case eval initialEnv expr of
        Left errMsg -> Left errMsg
        Right (val, _traceLog) -> Right val -- Discard trace log for
           simplicity in tests
    Just (expr, rest) -> Left ("TEST_ERROR: Parser did not consume
       entire input. Remainder: " ++ rest)
```

The tests were designed to cover:

- **Valid Inputs**: Ensuring that well-formed expressions, including function definitions and applications, parse and evaluate to their correct values.
- **Invalid Inputs (Parse Errors)**: Verify that the parser correctly rejects malformed expressions.
- **Runtime Errors**: Confirming that the evaluator correctly identifies and reports semantic errors (e.g., type mismatches, undefined variables, applying nonfunctions).

7.2 Test Coverage

A comprehensive suite of tests was developed. In addition to tests for literals, arithmetic, let, if, and basic list operations, significant attention was given to:

Listing 8: Helper function for parsing and evaluating input expressions

```
testEvalNum :: Test
testEvalNum = "evaluate number" ~: Right (VNum 42) ~=? evalTest "42"

testEvalNumZero :: Test
testEvalNumZero = "evaluate zero" ~: Right (VNum 0) ~=? evalTest "0"
```

Listing 9: Arithmetic operations tests

```
testAdd :: Test
testAdd = "evaluate␣1␣+␣2" ~: Right (VNum 3) ~=? evalTest "1␣+␣2"

testSub :: Test
testSub = "evaluate␣5␣-␣2" ~: Right (VNum 3) ~=? evalTest "5␣-␣2"

testMul :: Test
testMul = "evaluate␣3␣*␣4" ~: Right (VNum 12) ~=? evalTest "3␣*␣4"

testArithmeticPrecedence :: Test
testArithmeticPrecedence = "evaluate␣1␣+␣2␣*␣3␣(precedence)" ~: Right (
    VNum 7) ~=? evalTest "1␣+␣2␣*␣3"

testArithmeticParentheses :: Test
testArithmeticParentheses = "evaluate␣(1␣+␣2)␣*␣3␣(parentheses)" ~:
    Right (VNum 9) ~=? evalTest "(1␣+␣2)␣*␣3"

testArithmeticLeftAssociativity :: Test
testArithmeticLeftAssociativity = "evaluate␣10␣-␣3␣-␣2␣(left-
    associativity)" ~: Right (VNum 5) ~=? evalTest "10␣-␣3␣-␣2"
```

Listing 10: Equality operation tests

```
testEqNumTrue :: Test
testEqNumTrue = "evaluate␣5␣==␣5" ~: Right (VBool True) ~=? evalTest "5
    ␣==␣5"

testEqNumFalse :: Test
testEqNumFalse = "evaluate␣5␣==␣3" ~: Right (VBool False) ~=? evalTest
    "5␣==␣3"

testEqBoolTrue :: Test
testEqBoolTrue = "evaluate␣True␣==␣True" ~: Right (VBool True) ~=?
    evalTest "True␣==␣True"

testEqBoolFalse :: Test
testEqBoolFalse = "evaluate␣True␣==␣False" ~: Right (VBool False) ~=?
    evalTest "True␣==␣False"

testEqMixedTypes :: Test
testEqMixedTypes = "evaluate␣1␣==␣False␣(mixed␣types)" ~: Right (VBool
    False) ~=? evalTest "1␣==␣False"

testEqEmptyLists :: Test
testEqEmptyLists = "evaluate␣[]␣==␣[]" ~: Right (VBool True) ~=?
    evalTest "[]␣==␣[]"

testEqListsTrue :: Test
testEqListsTrue = "evaluate␣[1,2]␣==␣[1,2]" ~: Right (VBool True) ~=?
    evalTest "[1,2]␣==␣[1,2]" -- Assuming parser takes comma separated

testEqListsFalse :: Test
testEqListsFalse = "evaluate␣[1,2]␣==␣[1,3]" ~: Right (VBool False) ~=?
     evalTest "[1,2]␣==␣[1,3]"
```

Listing 11: Let expression tests

```
testLetSimple :: Test
testLetSimple = "evaluate␣let␣x␣=␣10␣in␣x" ~: Right (VNum 10) ~=?
    evalTest "let␣x␣=␣10␣in␣x"

testLetCalculation :: Test
testLetCalculation = "evaluate␣let␣x␣=␣5␣in␣x␣+␣x" ~: Right (VNum 10)
    ~=? evalTest "let␣x␣=␣5␣in␣x␣+␣x"

testLetNested :: Test
testLetNested = "evaluate␣let␣x␣=␣3␣in␣let␣y␣=␣4␣in␣x␣+␣y" ~: Right (
    VNum 7) ~=? evalTest "let␣x␣=␣3␣in␣let␣y␣=␣4␣in␣x␣+␣y"

testLetShadowing :: Test
testLetShadowing = "evaluate␣let␣x␣=␣1␣in␣let␣x␣=␣2␣in␣x" ~: Right (
    VNum 2) ~=? evalTest "let␣x␣=␣1␣in␣let␣x␣=␣2␣in␣x"

testLetVarUndefinedInBody :: Test
testLetVarUndefinedInBody = "let␣x␣=␣5␣in␣y␣(y␣undefined)" ~: Left "
    Undefined␣variable:␣y" ~=? evalTest "let␣x␣=␣5␣in␣y"
```

Listing 12: Type error tests

```
testAddTypeErrNumBool :: Test
testAddTypeErrNumBool = "evaluate␣1␣+␣True␣(type␣error)" ~: Left "Type␣
    Error:␣+␣expects␣numbers.␣Got␣1␣and␣True" ~=? evalTest "1␣+␣True"

testAddTypeErrBoolNum :: Test
testAddTypeErrBoolNum = "evaluate␣False␣+␣2␣(type␣error)" ~: Left "Type
    ␣Error:␣+␣expects␣numbers.␣Got␣False␣and␣2" ~=? evalTest "False␣+␣2"

testMulTypeErrBoolBool :: Test
testMulTypeErrBoolBool = "evaluate␣True␣*␣False␣(type␣error)" ~: Left "
    Type␣Error:␣*␣expects␣numbers.␣Got␣True␣and␣False" ~=? evalTest "
    True␣*␣False"
```

Listing 13: Complex expressions

```
testComplexLetIfArith :: Test
testComplexLetIfArith = "complex␣let/if/arithmetic" ~: Right (VNum 15)
    ~=? evalTest "let␣x␣=␣10␣in␣if␣x␣==␣10␣then␣(let␣y␣=␣x␣*␣2␣in␣y␣-␣5)
    ␣else␣0"
```

This structured approach aimed to cover not only "happy path" scenarios but also common errors and edge cases related to function semantics.

7.2.1 Coverage results

As shown below, all our unit tests ran successfully without failures. The image below 7 displays the total number of test cases (52), number of tests ran (52), errors encountered while running the tests (0) and failed test cases (0).



Figure 7: Test coverage results.

7.3 Evaluation of Results

After implementing and testing the interpreter, we achieved the following results.

- **Correctness**: The interpreter successfully implements the defined language features, including first-class, lexically scoped user-defined functions. The `HUnit` test suite (`Spec.hs`) verifies that lambda abstractions create correct closures, and function applications evaluate as expected, with respect to the lexical scope and handling of currying. The REPL in `Main.hs` further demonstrates support for named recursive functions. Trace logs confirm step-by-step execution, including environment extensions during function calls.
- **Robustness**: The interpreter handles syntax errors during parsing. During evaluation, it catches runtime errors such as type mismatches (e.g., applying a number as a function), undefined variables (including undefined functions), and incorrect argument counts if arity checking were more explicit (currently relies on currying).
- **Clarity and maintainability**: The Haskell codebase is modular. The implementation of closures in `Value.hs` and their handling in the `Lam` and `App` cases of `Evaluator.hs` are central to the functional capabilities and are relatively clear due to Haskell's features.
- **Performance**: Performance is adequate for the intended scope. Function calls involve environment manipulation and recursive calls to `eval`, which is standard for this type of interpreter. No specific performance optimizations for function calls (such as tail-call optimization, unless provided inherently by GHC for certain patterns) were implemented.

7.3.1 Comparison with related work

Table 1: Feature Comparison of Project `L`, AskElle, and Haskell Playground

Feature	**Project `L`**	**AskElle**	**Haskell Playground**
Evaluation Trace	✓	–	–
Save Snippets	✓	–	–
AST View	✓	–	–
Environment View	✓	–	–
Run / Evaluate	✓	✓	✓
REPL / CLI	✓	–	–

The tools developed for `L`, namely the Web User Interface and the REPL, provide a comprehensive set of educational and debugging features not found in comparable platforms like AskElle and Haskell Playground. It uniquely supports evaluation tracing, AST and environment views, saving snippets, and a REPL/CLI interface. The only common feature among all three tools is the execution of valid *L* programs. This highlights an emphasis on transparency and interactivity during evaluation.

7.4 Limitations

Despite its successes, the current interpreter has several limitations.

- **No Explicit let rec for expressions**: While the REPL/Web UI supports recursive definitions at the top level using a Haskell-specific lazy evaluation, the language itself lacks an explicit `let rec <name> = <lambda> in <expr>` construct for defining local recursive functions directly within expressions in a way that *eval* alone would handle without the `Main.hs` setup. The standard let is nonrecursive for the binding itself.
- **Basic Error Reporting**: Error messages, while functional, lack source code location information (line/column numbers), which is particularly helpful for debugging complex function calls or definitions.
- **Dynamic Typing**: Type checking is performed dynamically during evaluation. There is no separate static-type checking phase. This means that type errors in function arguments or return values are only caught at runtime.
- **Limited Standard Library**: The language offers a minimal set of built-in primitives.
- **No I/O Operations**: The language in its current setting cannot perform input/output operations.

8 Conclusion

This project successfully achieved its primary aim: the design and implementation of an SOS-based interpreter for a custom-defined functional language using Haskell. Key achievements include:

- **Complete Interpretation Pipeline for a Functional Language *L***: A complete pipeline supporting function definition (lambda abstractions), closure creation, and function application was realized.
- **Core Functional Language Features Implemented**: Based on the specified SOS inference rules, the interpreter correctly handles fundamental constructs including integer/boolean literals, arithmetic, equality, let bindings, if-then-else, list operations, and, critically, user-defined functions with lexical scope and currying. The named recursive functions are supported at the top-level in the REPL.
- **Error handling**: Simple mechanisms for detecting and reporting parse-time syntax errors and run-time semantic errors (type mismatches, undefined variables, application of nonfunctions) were integrated.
- **Haskell for interpreters**: The project effectively showcased Haskell's strengths, particularly ADTs for Expr and Value (including VClosure), pattern matching, and monads.

- **Testing**: A comprehensive HUnit test suite verified the correctness of implemented features, including various scenarios of function definition and application. The project provided significant practical experience in the theory and application of programming language interpretation, particularly concerning closures, environments, lexical scoping, and function call semantics.

8.1 Reflections

8.1.1 What went well?

- The implementation of lexical scoping via closures (`VClosure` capturing `Env`) was successful and is a cornerstone of the functional capabilities of the project.
- Haskell's features (ADTs, pattern matching, Map for environments) greatly facilitated a clean implementation of these complex semantics.
- The parser correctly handles lambda syntax and function application, including left-associativity for application.
- The support for top-level recursive function definitions is a valuable feature. Unit testing function-related behavior provided good confidence.

8.1.2 What could have been done differently?

- **Explicit let rec**: Consideration could have been given to implementing an explicit let rec syntax within the expression language itself, which would require a different handling in the eval function for Let (e.g., using a fixed-point combinator like the Y combinator, or a two-stage environment update). This would make local recursion more idiomatic within the language.
- **Error Reporting for Functions**: More detailed error messages for function calls (e.g., "function X expected Y arguments, got Z", even with currying, or better type mismatch details) would improve usability.

8.1.3 Comparison with objectives

The project successfully met all core objectives concerning the implementing an SOS-based interpreter for a target language L that is an untyped functional language, including user-defined functions.

8.1.4 Lessons Learned

- Implementing closures and lexical scope correctly is intricate and requires careful attention to environment management at definition time and application time.
- Testing various aspects of function calls (simple application, currying, higher-order functions, recursion, scope capture) is essential for validating a functional interpreter.
- Haskell's type system helps, but logical errors in environment handling or evaluation rules for functions can still be subtle.

8.2 Future Work

In this paper, we specified the syntax and semantics of an simple imperative language based on lambda calculus. We have built a solid foundation that is highly extensible and can be improved to a much more complete language with extra functional features.

Moving forward, our goal is to enhance this implementation of L in several key areas:

- **Explicit let rec for expressions**: Add syntax and evaluation semantics for `let rec f = -> ... f ... in ...` to allow local recursive function definitions directly within expressions, independent of the REPL's top-level mechanism.
- **Static Type System and Type Inference**: This remains a major and valuable extension. Implement a type checker (e.g. Hindley-Milner) to analyze the AST and infer/check types before evaluation, catching type errors statically. This would involve defining type environments and type rules for all expressions, including function types (TFun TArg TResult).
- **Pattern Matching**: Introduce case expressions or pattern matching in function definitions for deconstructing values (especially lists and future ADTs).
- **More Sophisticated Error Reporting**: Include line/column numbers and more descriptive error messages, especially for type errors related to function arguments/return values.
- **Expanded Data Types**: Strings, user-defined ADTs.
- **Compilation**: Transform the interpreter into a compiler that targets bytecode or another intermediate representation.

These extensions would further enhance the language's expressiveness, safety, and practicality.

9 References

References

[1] G. D. Plotkin, "A structural approach to operational semantics," *DAIMI Report FN-19*, Aarhus University, 1981.

[2] M. Hennessy, *The Semantics of Programming Languages: An Elementary Introduction*, Wiley, 1990.

[3] P. D. Mosses, "Denotational semantics and operational semantics: A modular approach," *Acta Informatica*, vol. 27, no. 5, pp. 497–528, 1990.

[4] G. Winskel, *The Formal Semantics of Programming Languages: An Introduction*, MIT Press, 1993.

[5] P. Hudak, "A gentle introduction to Haskell," *Journal of Functional Programming*, vol. 17, no. 4–5, pp. 567–604, 2007.

[6] G. Hutton, *Programming in Haskell*, 2nd ed., Cambridge University Press, 2016.

[7] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006.

[8] D. Leijen and E. Meijer, "Parsec: Direct-style monadic parser combinators for the real world," *Technical Report UU-CS-2001-27*, Utrecht University, 2001.

[9] B. C. Pierce, *Types and Programming Languages*, MIT Press, 2002.

[10] H. R. Nielson and F. Nielson, *Semantics with Applications*, Wiley, 1992.

[11] M. Felleisen, R. Findler, M. Flatt, and S. Krishnamurthi, *Semantics Engineering with PLT Redex*, MIT Press, 2009.

[12] G. Kahn, "Natural semantics," in *Proc. Symp. Theoretical Aspects of Computer Science (STACS)*, 1987, pp. 22–39.

[13] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[14] *parsec: Monadic parser combinators*, Hackage, accessed Apr. 28, 2025. Available: `https://hackage.haskell.org/package/parsec`

[15] *Megaparsec tutorial*, M. Karpov, accessed Apr. 28, 2025. Available: `https://markkarpov.com/tutorial/megaparsec.html`

[16] H. C. Cunningham, "Chapter 42: Abstract Syntax," *Explorations in Language Implementation*, 2018. Available: `https://john.cs.olemiss.edu/~hcc/csci450/ELIFP/Ch42/42_Abstract_Syntax.html`

[17] T. Austin, "Big-step operational semantics," *CS 252 Lecture Slides*, San Jose State University, Spring 2022. [Online]. Available: `http://www.cs.sjsu.edu/~austin/cs252-spring22/slides/CS252-Day04_BigStepOperationalSemantics.pdf`

[18] R. Ray, "Operational semantics," accessed Apr. 28, 2025. [Online]. Available: `https://cs.lmu.edu/~ray/notes/opsem/`

[19] PLS Lab, "Structural operational semantics," accessed Apr. 28, 2025. [Online]. Available: `https://www.pls-lab.org/Structural_Operational_Semantics`

[20] D. Chappell, "Specifying semantics: how interpreters work," University of Alaska Fairbanks, Department of Computer Science. Accessed Apr. 28, 2025. [Online]. Available: `https://www.cs.uaf.edu/~chappell/class/2025_spr/cs331/lect/cs331-20250402-interp_work.pdf`

[21] Bookish Press, "Human-centered programming languages: operational semantics," accessed Apr. 28, 2025. [Online]. Available: `https://bookish.press/hcpl/chapter14`

[22] University of Cambridge, "Semantics of programming languages," accessed Apr. 28, 2025. [Online]. Available: `https://www.cl.cam.ac.uk/teaching/1213/Semantics/notes-mono.pdf`

[23] A. Church, "An unsolvable problem of elementary number theory," *American Journal of Mathematics*, vol. 58, no. 2, pp. 345–363, 1936.

[24] H. P. Barendregt, *The Lambda Calculus: Its Syntax and Semantics*, North Holland, 1984.

[25] G. D. Plotkin, "Call-by-name, call-by-value and the lambda-calculus," *Theoretical Computer Science*, vol. 1, no. 2, pp. 125–159, 1975.

[26] S. L. Peyton Jones, *The Implementation of Functional Programming Languages*, Prentice-Hall, 1987.

[27] Z. M. Ariola and M. Felleisen, "An operational semantics for call-by-need," *Information and Computation*, vol. 117, no. 1, pp. 62–103, 1995.

[28] Z. M. Ariola and M. Felleisen, "Call by need," *Journal of Functional Programming*, vol. 7, no. 3, pp. 265–301, 1997.